



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ
КАФЕДРА СИСТЕМЫ ОБРАБОТКИ ИНФОРМАЦИИ И
УПРАВЛЕНИЯ

ДОМАШНЯЯ РАБОТА №2

«Разработка консольного приложения с применением ООП»

ДИСЦИПЛИНА: «Архитектура автоматизированных систем обработки информации и управления»

Выполнил: студент гр. ИУ5-22Б

(Подпись) (Краснова А. А.)
(Ф.И.О.)

Проверил:

(Подпись) (Гапанюк О. Е.)
(Ф.И.О.)

Дата сдачи (защиты):

Результаты сдачи (защиты):

- Балльная оценка:

- Оценка:

Москва, 2026 г.

Файл Publisher.cs - класс издательства (справочник):

```
namespace Homework2
{
    public class Journal
    {
        public int Id { get; set; }
        public int PublisherId { get; set; }
        public string Name { get; set; }
        private int _circulationK;
        public int CirculationK
        {
            get => _circulationK;
            set
            {
                if (value < 0)
                    throw new ArgumentException("Тираж не может быть отрицательным!");
                _circulationK = value;
            }
        }
        public Journal(int id, int publisherId, string name, int circulationK)
        {
            Id = id;
            PublisherId = publisherId;
            Name = name;
            CirculationK = circulationK;
        }
        public Journal() : this(0, 0, "", 0) { }
        public override string ToString() => $"[{Id}] {Name}, изд-во #{PublisherId},
тираж: {CirculationK} тыс. экз.";
    }
}
```

Файл Journal.cs - класс журнала (основная таблица):

```
namespace Homework2
{
    public class Journal
    {
        public int Id { get; set; }
        public int PublisherId { get; set; }
        public string Name { get; set; }
        private int _circulationK;
        public int CirculationK
        {
            get => _circulationK;
            set
            {
                if (value < 0)
                    throw new ArgumentException("Тираж не может быть отрицательным!");
                _circulationK = value;
            }
        }
        public Journal(int id, int publisherId, string name, int circulationK)
        {
            Id = id;
            PublisherId = publisherId;
            Name = name;
            CirculationK = circulationK;
        }
        public Journal() : this(0, 0, "", 0) { }
        public override string ToString() => $"[{Id}] {Name}, изд-во #{PublisherId},
тираж: {CirculationK} тыс. экз.";
    }
}
```

Файл DatabaseManager.cs:

```

using Microsoft.Data.Sqlite;

namespace Homework2
{
    class DatabaseManager
    {
        private readonly string _connectionString;

        public DatabaseManager(string dbPath)
        {
            _connectionString = $"Data Source={dbPath}";
            InitializeDatabase();
        }

        private void InitializeDatabase()
        {
            using var connection = new SqliteConnection(_connectionString);
            connection.Open();

            var command = connection.CreateCommand();
            command.CommandText = @"
                CREATE TABLE IF NOT EXISTS publisher (
                    publisher_id INTEGER PRIMARY KEY,
                    publisher_name TEXT NOT NULL
                );

                CREATE TABLE IF NOT EXISTS journal (
                    journal_id INTEGER PRIMARY KEY,
                    publisher_id INTEGER NOT NULL,
                    journal_name TEXT NOT NULL,
                    circulation_k INTEGER NOT NULL,
                    FOREIGN KEY (publisher_id) REFERENCES publisher(publisher_id)
                );";
            command.ExecuteNonQuery();
        }

        public void ImportFromCsv(string publishersPath, string journalsPath)
        {
            ImportPublishersFromCsv(publishersPath);
            ImportJournalsFromCsv(journalsPath);
        }

        private void ImportPublishersFromCsv(string path)
        {
            if (!File.Exists(path))
            {
                Console.WriteLine($"Файл {path} не найден!");
                return;
            }

            using var connection = new SqliteConnection(_connectionString);
            connection.Open();

            var checkCmd = connection.CreateCommand();
            checkCmd.CommandText = "SELECT COUNT(*) FROM publisher";
            long count = (long)checkCmd.ExecuteScalar();

            if (count > 0)
                return;

            var lines = File.ReadAllLines(path);

            for (int i = 1; i < lines.Length; i++)
            {
                if (string.IsNullOrWhiteSpace(lines[i]))
                    continue;

                var parts = lines[i].Split(';');
            }
        }
    }
}

```

```

        if (parts.Length < 2)
            continue;

        var command = connection.CreateCommand();
        command.CommandText = "INSERT INTO publisher (publisher_id,
publisher_name) VALUES (@id, @name)";
        command.Parameters.AddWithValue("@id", int.Parse(parts[0]));
        command.Parameters.AddWithValue("@name", parts[1]);
        command.ExecuteNonQuery();
    }
}

private void ImportJournalsFromCsv(string path)
{
    if (!File.Exists(path))
    {
        Console.WriteLine($"Файл {path} не найден!");
        return;
    }

    using var connection = new SqlConnection(_connectionString);
    connection.Open();

    var checkCmd = connection.CreateCommand();
    checkCmd.CommandText = "SELECT COUNT(*) FROM journal";
    long count = (long)checkCmd.ExecuteScalar();

    if (count > 0)
        return;

    var lines = File.ReadAllLines(path);

    for (int i = 1; i < lines.Length; i++)
    {
        if (string.IsNullOrEmpty(lines[i]))
            continue;

        var parts = lines[i].Split(';');

        if (parts.Length < 4)
            continue;

        var command = connection.CreateCommand();
        command.CommandText = @"
            INSERT INTO journal (journal_id, publisher_id, journal_name,
circulation_k)
            VALUES (@id, @pubId, @name, @circ)";
        command.Parameters.AddWithValue("@id", int.Parse(parts[0]));
        command.Parameters.AddWithValue("@pubId", int.Parse(parts[1]));
        command.Parameters.AddWithValue("@name", parts[2]);
        command.Parameters.AddWithValue("@circ", int.Parse(parts[3]));
        command.ExecuteNonQuery();
    }
}

public List<Publisher> GetAllPublishers()
{
    var result = new List<Publisher>();
    using var connection = new SqlConnection(_connectionString);
    connection.Open();

    var command = connection.CreateCommand();
    command.CommandText = "SELECT publisher_id, publisher_name FROM publisher
ORDER BY publisher_id";

    using var reader = command.ExecuteReader();
    while (reader.Read())
    {

```

```

        result.Add(new Publisher(reader.GetInt32(0), reader.GetString(1)));
    }
    return result;
}

public List<Journal> GetAllJournals()
{
    var result = new List<Journal>();
    using var connection = new SqlConnection(_connectionString);
    connection.Open();

    var command = connection.CreateCommand();
    command.CommandText = "SELECT journal_id, publisher_id, journal_name,
circulation_k FROM journal ORDER BY journal_id";

    using var reader = command.ExecuteReader();
    while (reader.Read())
    {
        result.Add(new Journal(
            reader.GetInt32(0),
            reader.GetInt32(1),
            reader.GetString(2),
            reader.GetInt32(3)
        ));
    }
    return result;
}

public Journal? GetJournalById(int id)
{
    using var connection = new SqlConnection(_connectionString);
    connection.Open();

    var command = connection.CreateCommand();
    command.CommandText = "SELECT journal_id, publisher_id, journal_name,
circulation_k FROM journal WHERE journal_id = @id";
    command.Parameters.AddWithValue("@id", id);

    using var reader = command.ExecuteReader();
    if (reader.Read())
    {
        return new Journal(
            reader.GetInt32(0),
            reader.GetInt32(1),
            reader.GetString(2),
            reader.GetInt32(3)
        );
    }
    return null;
}

public void AddJournal(Journal journal)
{
    using var connection = new SqlConnection(_connectionString);
    connection.Open();

    var command = connection.CreateCommand();
    command.CommandText = @"
        INSERT INTO journal (publisher_id, journal_name, circulation_k)
        VALUES (@pubId, @name, @circ)";
    command.Parameters.AddWithValue("@pubId", journal.PublisherId);
    command.Parameters.AddWithValue("@name", journal.Name);
    command.Parameters.AddWithValue("@circ", journal.CirculationK);
    command.ExecuteNonQuery();
}

public void UpdateJournal(Journal journal)
{
    using var connection = new SqlConnection(_connectionString);

```

```

        connection.Open();

        var command = connection.CreateCommand();
        command.CommandText = @"
            UPDATE journal
            SET publisher_id = @pubId, journal_name = @name, circulation_k = @circ
            WHERE journal_id = @id";
        command.Parameters.AddWithValue("@id", journal.Id);
        command.Parameters.AddWithValue("@pubId", journal.PublisherId);
        command.Parameters.AddWithValue("@name", journal.Name);
        command.Parameters.AddWithValue("@circ", journal.CirculationK);
        command.ExecuteNonQuery();
    }

    public void DeleteJournal(int id)
    {
        using var connection = new SqlConnection(_connectionString);
        connection.Open();

        var command = connection.CreateCommand();
        command.CommandText = "DELETE FROM journal WHERE journal_id = @id";
        command.Parameters.AddWithValue("@id", id);
        command.ExecuteNonQuery();
    }

    public (string[] columns, List<string[]> rows) ExecuteQuery(string sql)
    {
        using var connection = new SqlConnection(_connectionString);
        connection.Open();

        var command = connection.CreateCommand();
        command.CommandText = sql;

        using var reader = command.ExecuteReader();

        var columns = new string[reader.FieldCount];
        for (int i = 0; i < reader.FieldCount; i++)
        {
            columns[i] = reader.GetName(i);
        }

        var rows = new List<string[]>();
        while (reader.Read())
        {
            var row = new string[reader.FieldCount];
            for (int i = 0; i < reader.FieldCount; i++)
            {
                row[i] = reader.GetValue(i)?.ToString() ?? "";
            }
            rows.Add(row);
        }
        return (columns, rows);
    }
}

```

Файл ReportBuilder.cs:

```

using System.Text;

namespace Homework2
{
    class ReportBuilder
    {
        private DatabaseManager _db;
        private string _sql = "";
        private string _title = "";
        private string[] _headers = Array.Empty<string>();
        private int[] _widths = Array.Empty<int>();
        private bool _numbered = false;
    }
}

```

```

private string _footer = "";

public ReportBuilder(DatabaseManager db)
{
    _db = db;
}

public ReportBuilder Query(string sql)
{
    _sql = sql;
    return this;
}

public ReportBuilder Title(string title)
{
    _title = title;
    return this;
}

public ReportBuilder Header(params string[] columns)
{
    _headers = columns;
    return this;
}

public ReportBuilder ColumnWidths(params int[] widths)
{
    _widths = widths;
    return this;
}

public ReportBuilder Numbered()
{
    _numbered = true;
    return this;
}

public ReportBuilder Footer(string label)
{
    _footer = label;
    return this;
}

public string Build()
{
    {
        var (columns, rows) = _db.ExecuteQuery(_sql);
        var sb = new StringBuilder();

        if (_title.Length > 0)
        {
            sb.AppendLine();
            sb.AppendLine($"=== {_title} ===");
        }

        string[] displayHeaders = _headers.Length > 0 ? _headers : columns;
        int colCount = displayHeaders.Length;
        int[] widths;

        if (_widths.Length >= colCount)
        {
            widths = _widths;
        }
        else
        {
            widths = new int[colCount];
            for (int i = 0; i < colCount; i++)
                widths[i] = 20;
        }
    }
}

```

```

        int numWidth = _numbered ? 5 : 0;

        if (_numbered)
            sb.Append("%".PadRight(numWidth));

        for (int i = 0; i < colCount; i++)
            sb.Append(displayHeaders[i].PadRight(widths[i]));
        sb.AppendLine();

        int totalWidth = numWidth;
        for (int i = 0; i < colCount; i++)
            totalWidth += widths[i];
        sb.AppendLine(new string('-', totalWidth));

        for (int r = 0; r < rows.Count; r++)
        {
            if (_numbered)
                sb.Append((r + 1).ToString().PadRight(numWidth));

            for (int c = 0; c < rows[r].Length && c < colCount; c++)
                sb.Append(rows[r][c].PadRight(widths[c]));
            sb.AppendLine();
        }

        if (_footer.Length > 0)
        {
            sb.AppendLine(new string('-', totalWidth));
            sb.AppendLine($"{_footer}: {rows.Count}");
        }

        return sb.ToString();
    }
    public void Print()
    {
        Console.WriteLine(Build());
    }
    public void SaveToFile(string path)
    {
        File.WriteAllText(path, Build());
        Console.WriteLine($"Отчёт сохранён в файл: {path}");
    }
}
}

```

Файл Program.cs - основная программа:

```

using Homework2;
using System.Text;

//Код взят из методических указаний и адаптирован под мой вариант
// Устанавливаем кодировку для корректного отображения русского языка
Console.OutputEncoding = Encoding.UTF8;
Console.InputEncoding = Encoding.UTF8;

// Пути к файлам
string dbPath = "publishers_journals.db";
string publishersCsv = Path.Combine(AppContext.BaseDirectory, "publishers.csv");
string journalsCsv = Path.Combine(AppContext.BaseDirectory, "journals.csv");

// Создаём менеджер БД и импортируем данные из CSV
var db = new DatabaseManager(dbPath);
db.ImportFromCsv(publishersCsv, journalsCsv);

Console.WriteLine();
Console.WriteLine("
Console.WriteLine("
Console.WriteLine("
Console.WriteLine("
");
");
");
");
");

// Главный цикл меню

```

```
string choice;
do
{
    Console.WriteLine(" ");
    Console.WriteLine("          ГЛАВНОЕ МЕНЮ");
    Console.WriteLine(" ");
    Console.WriteLine("   1 - Показать все издательства");
    Console.WriteLine("   2 - Показать все журналы");
    Console.WriteLine("   3 - Добавить журнал");
    Console.WriteLine("   4 - Редактировать журнал");
    Console.WriteLine("   5 - Удалить журнал");
    Console.WriteLine("   6 - Отчёты");
    Console.WriteLine("   0 - Выход");
    Console.WriteLine(" ");
    Console.Write("Ваш выбор: ");

    choice = Console.ReadLine()?.Trim() ?? "";
    Console.WriteLine();

    switch (choice)
    {
        case "1": ShowPublishers(db); break;
        case "2": ShowJournals(db); break;
        case "3": AddJournal(db); break;
        case "4": EditJournal(db); break;
        case "5": DeleteJournal(db); break;
        case "6": ReportsMenu(db); break;
        case "0": Console.WriteLine("До свидания!"); break;
        default: Console.WriteLine("Неверный пункт меню."); break;
    }
    Console.WriteLine();
} while (choice != "0");

// =====
// Функции для работы с данными
// =====

static void ShowPublishers(DatabaseManager db)
{
    Console.WriteLine("================= ВСЕ ИЗДАТЕЛЬСТВА =================");
    var publishers = db.GetAllPublishers();
    foreach (var pub in publishers)
        Console.WriteLine(" " + pub);
    Console.WriteLine($"-----");
    Console.WriteLine($"Итого издательств: {publishers.Count}");
}

static void ShowJournals(DatabaseManager db)
{
    Console.WriteLine("================= ВСЕ ЖУРНАЛЫ =================");
    var journals = db.GetAllJournals();
    foreach (var jrn in journals)
        Console.WriteLine(" " + jrn);
    Console.WriteLine($"-----");
    Console.WriteLine($"Итого журналов: {journals.Count}");
}

static void AddJournal(DatabaseManager db)
{
    Console.WriteLine("================= ДОБАВЛЕНИЕ ЖУРНАЛА =================");

    // Показываем доступные издательства
    Console.WriteLine("Доступные издательства:");
    var publishers = db.GetAllPublishers();
    foreach (var pub in publishers)
        Console.WriteLine(" " + pub);
    Console.WriteLine();

    // Ввод ID издательства
```

```

Console.Write("Введите ID издательства: ");
if (!int.TryParse(Console.ReadLine(), out int publisherId))
{
    Console.WriteLine(" Ошибка: введите целое число.");
    return;
}

// Проверяем, что издательство существует
bool publisherExists = publishers.Any(p => p.Id == publisherId);
if (!publisherExists)
{
    Console.WriteLine("Ошибка: издательство с таким ID не найдено.");
    return;
}

// Ввод названия журнала
Console.Write("Введите название журнала: ");
string name = Console.ReadLine()?.Trim() ?? "";
if (string.IsNullOrEmpty(name))
{
    Console.WriteLine("Ошибка: название не может быть пустым.");
    return;
}

// Ввод тиража
Console.Write("Введите тираж (тыс. экземпляров): ");
if (!int.TryParse(Console.ReadLine(), out int circulation))
{
    Console.WriteLine("Ошибка: введите целое число.");
    return;
}

try
{
    var journal = new Journal(0, publisherId, name, circulation);
    db.AddJournal(journal);
    Console.WriteLine("Журнал успешно добавлен!");
}
catch (ArgumentException ex)
{
    Console.WriteLine($" Ошибка: {ex.Message}");
}
}

static void EditJournal(DatabaseManager db)
{
    Console.WriteLine("===== РЕДАКТИРОВАНИЕ ЖУРНАЛА =====");

    Console.Write("Введите ID журнала для редактирования: ");
    if (!int.TryParse(Console.ReadLine(), out int id))
    {
        Console.WriteLine("Ошибка: введите целое число.");
        return;
    }

    var journal = db.GetJournalById(id);
    if (journal == null)
    {
        Console.WriteLine($"Ошибка: журнал с ID={id} не найден.");
        return;
    }

    Console.WriteLine();
    Console.WriteLine($"Текущие данные: {journal}");
    Console.WriteLine();
    Console.WriteLine("(Нажмите Enter, чтобы оставить значение без изменений)");
    Console.WriteLine();

    // Редактирование названия

```

```

Console.Write($"Название журнала [{journal.Name}]: ");
string input = Console.ReadLine()?.Trim() ?? "";
if (!string.IsNullOrEmpty(input))
    journal.Name = input;

// Редактирование ID издательства
Console.Write($"ID издательства [{journal.PublisherId}]: ");
input = Console.ReadLine()?.Trim() ?? "";
if (!string.IsNullOrEmpty(input) && int.TryParse(input, out int newPublisherId))
{
    // Проверяем существование издательства
    var publishers = db.GetAllPublishers();
    if (publishers.Any(p => p.Id == newPublisherId))
        journal.PublisherId = newPublisherId;
    else
        Console.WriteLine(" Предупреждение: издательство с таким ID не найдено, ID
не изменён.");
}

// Редактирование тиража
Console.Write($"Тираж (тыс. экз.) [{journal.CirculationK}]: ");
input = Console.ReadLine()?.Trim() ?? "";
if (!string.IsNullOrEmpty(input) && int.TryParse(input, out int newCirculation))
{
    try
    {
        journal.CirculationK = newCirculation;
    }
    catch (ArgumentException ex)
    {
        Console.WriteLine($"Ошибка: {ex.Message}");
        return;
    }
}

db.UpdateJournal(journal);
Console.WriteLine("Данные журнала обновлены!");
}

static void DeleteJournal(DatabaseManager db)
{
    Console.WriteLine("===== УДАЛЕНИЕ ЖУРНАЛА =====");

    Console.Write("Введите ID журнала для удаления: ");
    if (!int.TryParse(Console.ReadLine(), out int id))
    {
        Console.WriteLine("Ошибка: введите целое число.");
        return;
    }

    var journal = db.GetJournalById(id);
    if (journal == null)
    {
        Console.WriteLine($"Ошибка: журнал с ID={id} не найден.");
        return;
    }

    Console.WriteLine();
    Console.WriteLine($"Вы действительно хотите удалить журнал?");
    Console.WriteLine($" {journal}");
    Console.WriteLine();
    Console.Write("Введите 'YES' для подтверждения: ");
    string confirm = Console.ReadLine()?.Trim() ?? "";
    if (confirm == "YES")
    {
        db.DeleteJournal(id);
        Console.WriteLine(" Журнал удалён.");
    }
    else

```

```
{
    Console.WriteLine("Удаление отменено.");
}

// =====
// Меню ОТЧЁТОВ
// =====

static void ReportsMenu(DatabaseManager db)
{
    string choice;
    do
    {
        Console.WriteLine("
        Console.WriteLine("
        Console.WriteLine("
        Console.WriteLine("
        Console.WriteLine("
        Console.WriteLine("
        Console.WriteLine("
        Console.WriteLine("
        Console.Write("Ваш выбор: ");

        choice = Console.ReadLine()?.Trim() ?? "";
        Console.WriteLine();

        switch (choice)
        {
            case "1": Report1_JournalsWithPublishers(db); break;
            case "2": Report2_CountByPublisher(db); break;
            case "3": Report3_AvgCirculationByPublisher(db); break;
            case "0": break;
            default: Console.WriteLine("Неверный пункт."); break;
        }
    } while (choice != "0");
}

// Отчёт 1: Журналы с названиями издательств (JOIN)
static void Report1_JournalsWithPublishers(DatabaseManager db)
{
    new ReportBuilder(db)
        .Query(@"
            SELECT j.journal_name, p.publisher_name, j.circulation_k
            FROM journal j
            JOIN publisher p ON j.publisher_id = p.publisher_id
            ORDER BY j.journal_name")
        .Title("ЖУРНАЛЫ ПО ИЗДАТЕЛЬСТВАМ")
        .Header("Название журнала", "Издательство", "Тираж (тыс. экз.)")
        .ColumnWidths(25, 20, 18)
        .Numbered()
        .Print();
}

// Отчёт 2: Количество журналов по издательствам (GROUP BY + COUNT)
static void Report2_CountByPublisher(DatabaseManager db)
{
    new ReportBuilder(db)
        .Query(@"
            SELECT p.publisher_name, COUNT(*) AS cnt
            FROM journal j
            JOIN publisher p ON j.publisher_id = p.publisher_id
            GROUP BY p.publisher_name
            ORDER BY cnt DESC")
        .Title("КОЛИЧЕСТВО ЖУРНАЛОВ ПО ИЗДАТЕЛЬСТВАМ")
        .Header("Издательство", "Количество журналов")
        .ColumnWidths(30, 20)
        .Footer("Всего журналов")
        .Print();
}
```

```

}

// Отчёт 3: Средний тираж по издательствам (GROUP BY + AVG)
static void Report3_AvgCirculationByPublisher(DatabaseManager db)
{
    Console.WriteLine("Выберите способ вывода отчёта:");
    Console.WriteLine("1 – Вывести на экран");
    Console.WriteLine("2 – Сохранить в файл");
    Console.Write("Ваш выбор: ");

    string choice = Console.ReadLine()?.Trim() ?? "";

    var report = new ReportBuilder(db)
        .Query(@"
            SELECT p.publisher_name, ROUND(AVG(j.circulation_k), 1) AS avg_circ
            FROM journal j
            JOIN publisher p ON j.publisher_id = p.publisher_id
            GROUP BY p.publisher_name
            ORDER BY avg_circ DESC")
        .Title("СРЕДНИЙ ТИРАЖ ПО ИЗДАТЕЛЬСТВАМ")
        .Header("Издательство", "Средний тираж (тыс. экз.)")
        .ColumnWidths(30, 25);

    if (choice == "2")
    {
        string filePath = Path.Combine(AppContext.BaseDirectory,
            "avg_circulation_report.txt");
        report.SaveToFile(filePath);
    }
    else
    {
        report.Print();
    }
}

```

Результаты работы программы:

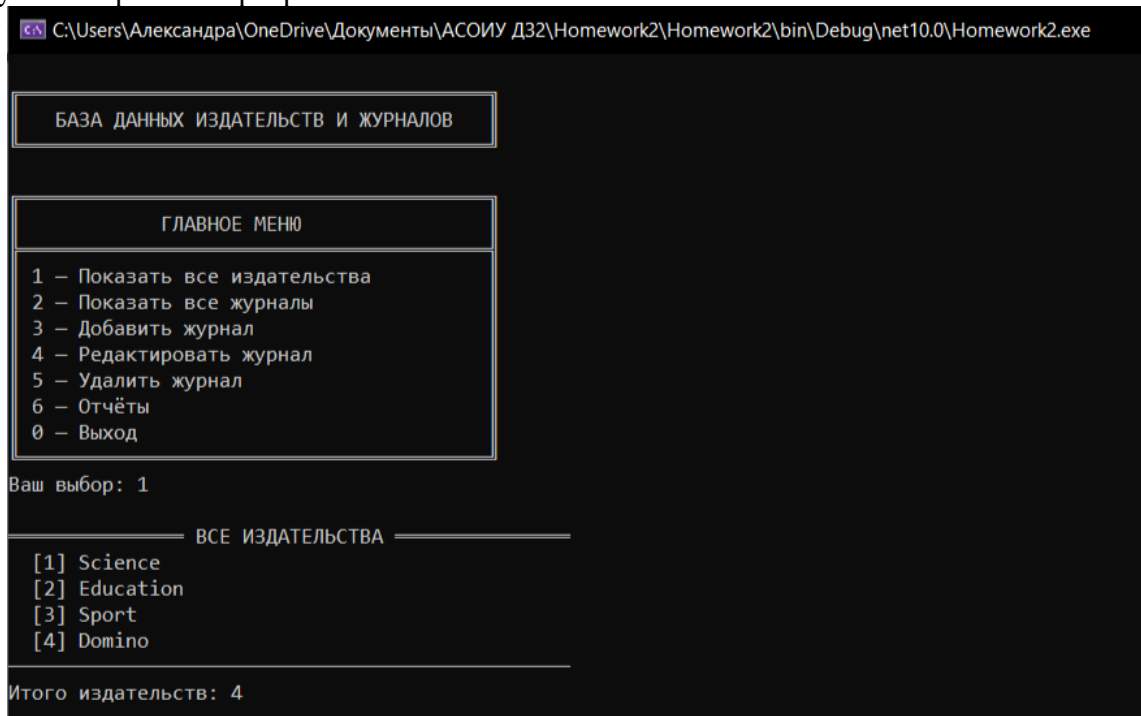


РИС.1

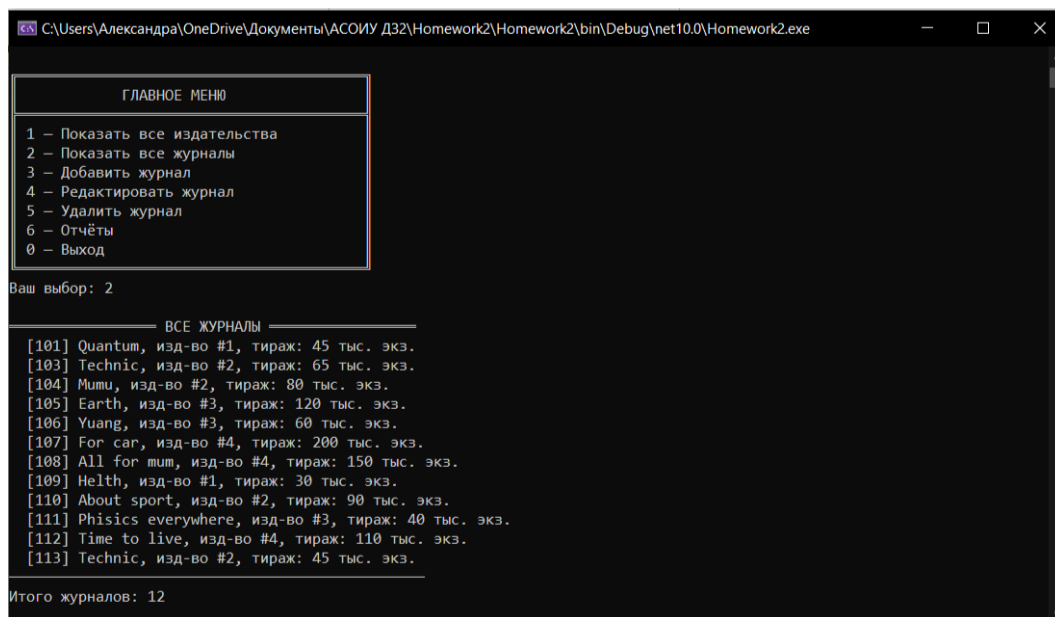


РИС.2

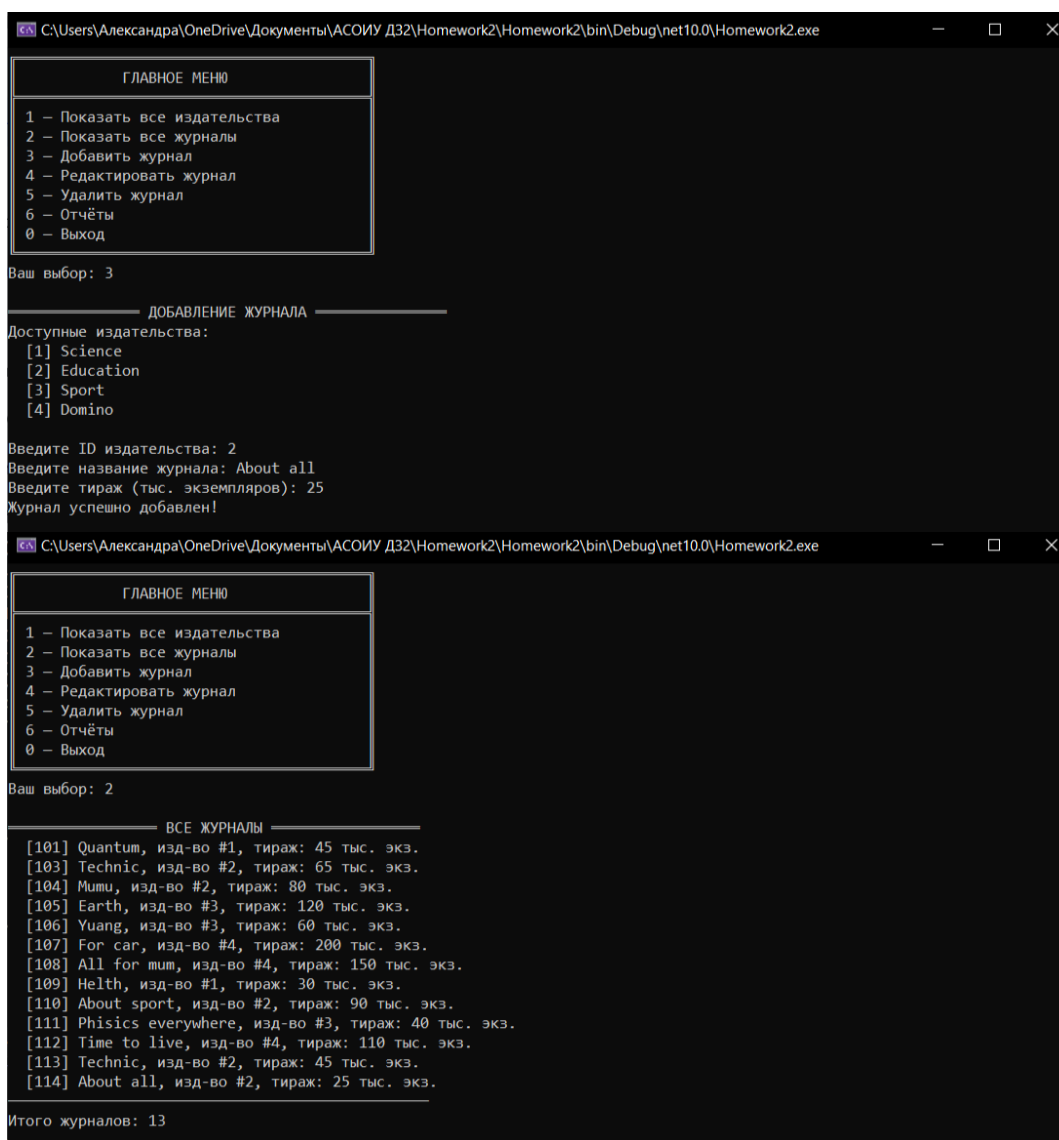


РИС.3

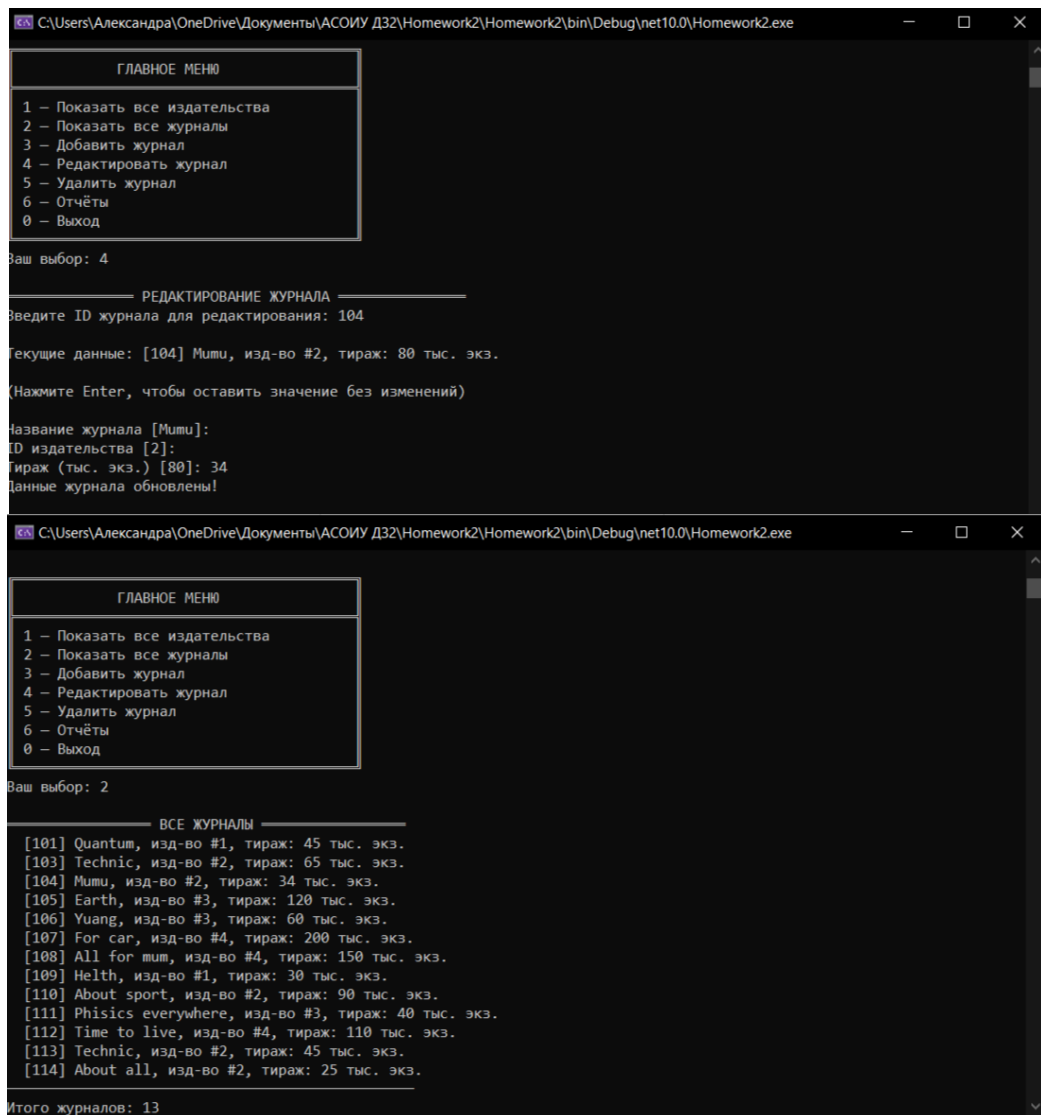
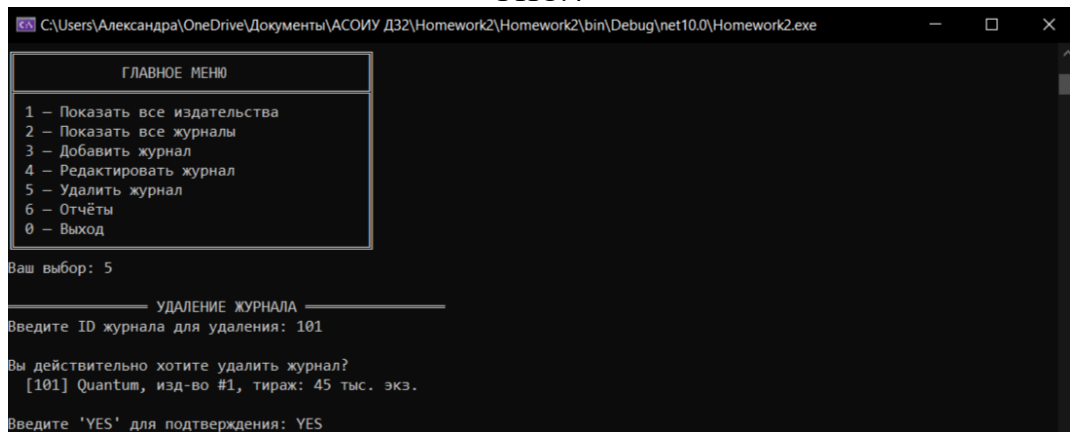


РИС.4



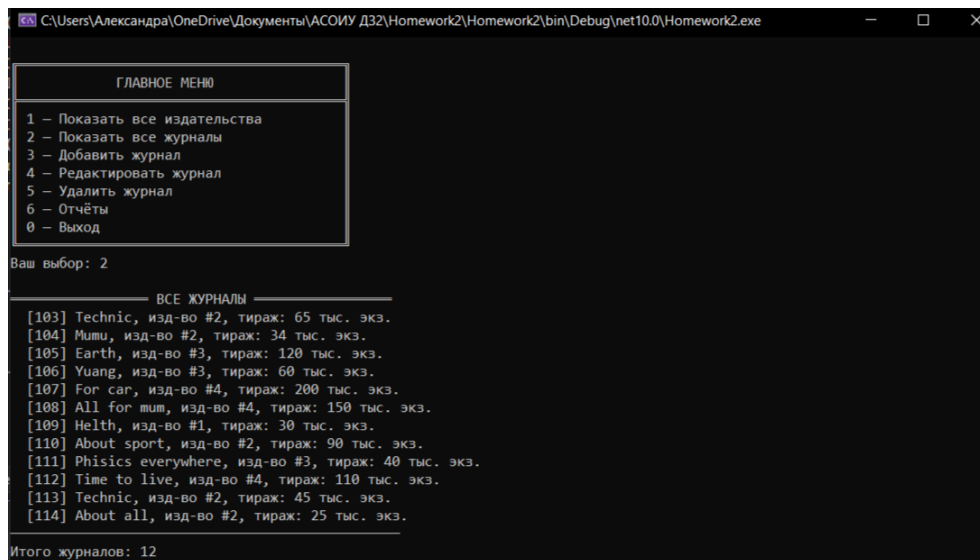


РИС.5

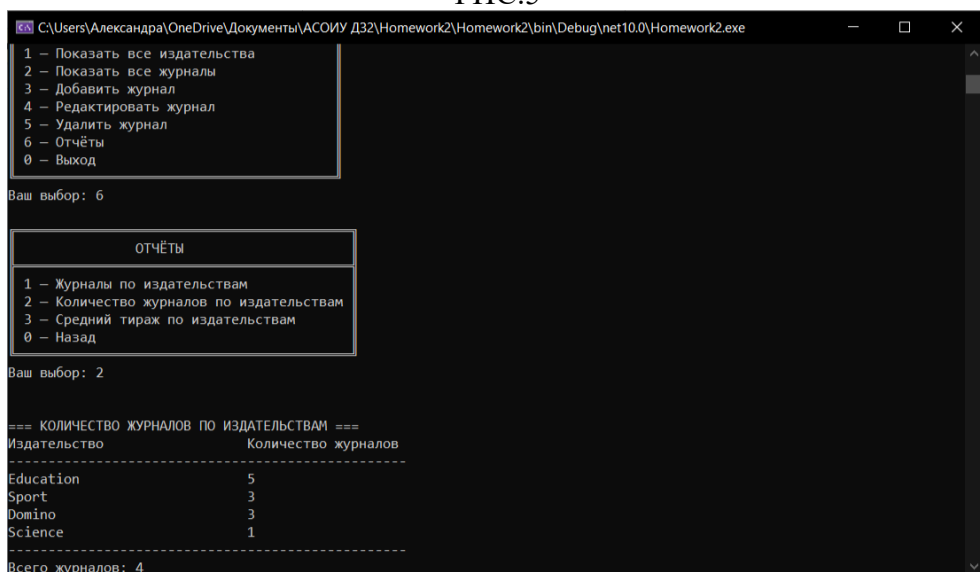


РИС.6

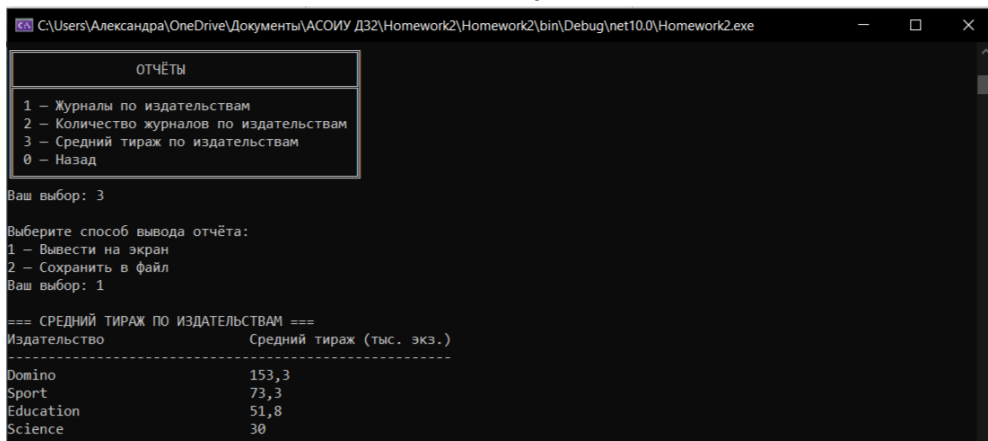


РИС.7

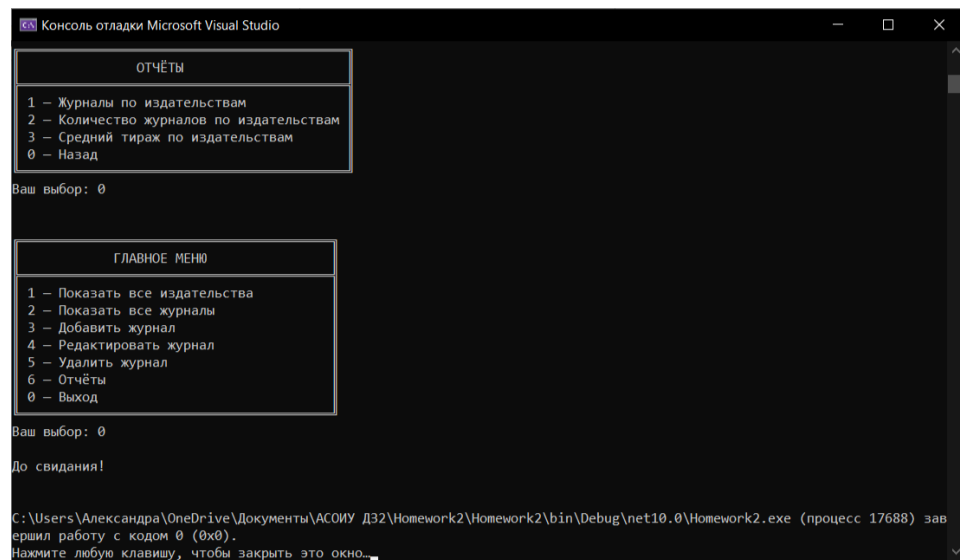


РИС.8

Схема А (модель данных)

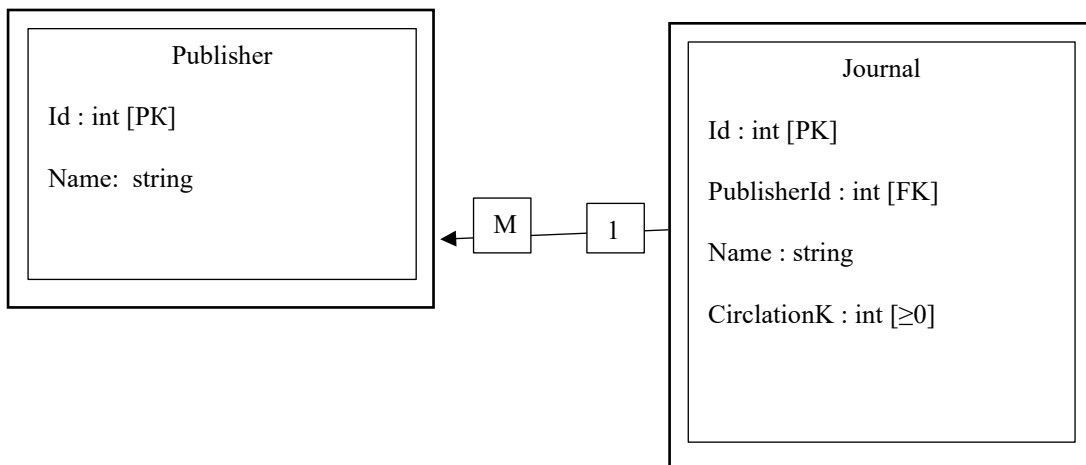


Схема Б (работы класса ReportBuilder)

